Shop Image Storage & CDN Setup

eSal backend — how we chose to store & serve shop images, what's been configured, and the upload flow left to build

Who this is for: written to walk a colleague (and future-us) through the reasoning behind every decision below, not just the steps — so it's clear *why* things are set up this way, not only *what* was clicked.

1. The Problem We're Solving

Digital receipts need to display the issuing shop's logo. Our Prisma schema already anticipated this: `Store.logoUrl` exists as a field and is already *read* by the dashboard and receipt endpoints — but nothing has ever *written* to it. There's no upload mechanism yet at all.

Separately, `Coupon.imageUrl` exists too — so image storage isn't a one-off need for shop logos, it's a shared capability multiple features will eventually rely on.

> **Open question we deliberately did not block on:** we don't yet know if end customers will ever be allowed to upload images themselves (vs. only shop owners, or only admins). We decided this doesn't matter for the storage design — the mechanism is identical either way. Only the permission check in front of the upload endpoint changes later; the storage/CDN plumbing is built once, now.

2. The Decision: Where Do Images Actually Live?

Two real options were on the table: **AWS S3 + CloudFront**, or **Cloudflare R2 + Cloudflare's CDN**. Both are "object storage" (a place to durably store files) that need a CDN layer in front of them to be fast and cheap at scale.

What a CDN actually does (background for anyone new to this)

Without a CDN, every request for an image travels all the way to one origin server, no matter where the requester is in the world. A **CDN** is a network of servers spread across many physical locations ("edge locations") that cache copies of files close to users. First request for a file near a location is a **cache miss** (fetched from the origin, then cached); every request after that, until the cache entry's **TTL** expires, is a **cache hit** — served instantly from the edge, without ever touching the origin again.

This matters a lot for our exact access pattern: a shop logo is **written once, rarely changed, but read constantly** — every receipt view, by every customer of that shop. That's the textbook case a CDN is built for.

Why R2 over S3

	S3 + CloudFront	R2 + Cloudflare CDN
Storage	~$0.023/GB/month	~$0.015/GB/month, 10GB free every month, permanently
Egress (data served to users)	Free for 12 months, then ~$0.085/GB and growing with traffic forever after	**$0, always, no cap, no expiry**
Free tier	Time-limited (12 months from account creation)	Permanent, resets monthly
SDK / API	AWS SDK (`@aws-sdk/client-s3`)	Same SDK — R2 is S3-API-compatible

> **Why this tips toward R2:** our cost is dominated by *reads* (many customers viewing the same unchanging logo), not storage. Egress is exactly the meter S3 charges for and R2 doesn't — so R2's pricing shape matches our workload's shape. Since R2 speaks the same S3 API, this isn't a one-way door either: application code would barely change if we ever needed to move to S3 later.

3. What We've Already Set Up DONE

1. Created the R2 bucket — `esal-image`

Storage Class: **Standard** (not "Infrequent Access" — that tier is priced for objects touched less than once/month, and our logos are read constantly). Location: Automatic → Western Europe, the closest available R2 region to our target market. This is the actual origin storage — the bucket that holds the real image bytes.

2. Created a scoped R2 API Token — `esal-backend-upload`

Permission: **Object Read & Write** (not Admin). Scope: applied to the `esal-image` bucket only, not the whole account. This produced an **Access Key ID + Secret Access Key** pair — the credentials our backend will use to authenticate.

> **Why scoped, not admin:** least privilege. If this credential ever leaked (committed to git by accident, exposed in a log), the blast radius is limited to reading/writing objects in one bucket — not deleting buckets or touching account settings. This single token is the R2 equivalent of what an AWS tutorial does in three separate steps: create an IAM User, attach a Policy to it, then generate access keys for it.

3. Registered a domain — `esal-image-cdn.win`

Bought directly through **Cloudflare Registrar**, which meant it was automatically already using Cloudflare's nameservers — no manual DNS/nameserver migration step needed.

> **Why a domain was required at all:** a CDN can only cache/proxy traffic for hostnames whose DNS it controls. Our bucket needed a real hostname pointing at it for Cloudflare's edge network to be able to cache it — that hostname has to live under a domain we own.

Note: `.win` is a TLD with a known spam/abuse reputation on some blocklists — flagged as a small, accepted risk for now (some networks may block/flag it). Cheap to replace later if it ever causes real image-delivery problems.

4. Connected a Custom Domain — `cdn.esal-image-cdn.win` → the bucket

Cloudflare automatically created a **CNAME** DNS record and issued an SSL certificate for this subdomain, pointing it at the `esal-image` bucket. Status moves from *Initializing* to *Active* on its own (DNS propagation + certificate issuance) — no action required.

> **Why a subdomain, not the root domain:** keeps the root domain free for anything else later, and makes the address self-explanatory — anyone reading `cdn.esal-image-cdn.win` immediately knows what it's for.

This is the URL prefix: `https://cdn.esal-image-cdn.win/<object-key>` is now our real, CDN-cached, public image URL — this is what will populate `Store.logoUrl` going forward.

4. What's Left to Build NEXT

Goal: let a client (the shop-owner app) upload a logo image **directly to R2**, without the image bytes ever passing through our NestJS server.

Step A — Install dependencies

```
pnpm add @aws-sdk/client-s3 @aws-sdk/s3-request-presigner
```

> R2 is S3-API-compatible, so the standard AWS SDK works unmodified — we just point its `endpoint` config at R2's URL instead of AWS's. No R2-specific library needed.

Step B — Add environment variables

```
R2_ACCOUNT_ID=<from the R2 overview page>
R2_ACCESS_KEY_ID=<from the API token we created>
R2_SECRET_ACCESS_KEY=<from the API token we created>
R2_BUCKET_NAME=esal-image
R2_PUBLIC_URL=https://cdn.esal-image-cdn.win
```

Real values go in `.env` (gitignored); placeholder empty entries go in `.env.example` so the shape is documented without leaking secrets.

Step C — The upload flow (presigned URL pattern)

1. Client calls `POST /shop/store/logo/upload-url` (guarded the same way as the rest of `/shop`). Backend generates a unique object key, e.g. `stores/<storeId>/logo-<uuid>.png`, and returns a short-lived (~5 min) **presigned PUT URL** for that exact key — plus content-type/size constraints baked in.

↓

2. Client uploads the image bytes **directly to R2** using that presigned URL.
The bytes never touch our NestJS server — they go straight from the client device to R2's storage.

↓

3. Backend sets `Store.logoUrl` to the final CDN URL: `https://cdn.esal-image-cdn.win/stores/<storeId>/logo-<uuid>.png`.
Since the key is known upfront, this can happen as soon as step 1 runs, or via a small "confirm" call after upload succeeds.

> **Why direct-to-storage, not proxied through our server:** Nest's default body parsing buffers uploads into server memory. Routing bytes straight to R2 instead means zero extra load on our app server no matter how many uploads happen concurrently, and it's the same amount of work to build either way.

Step D — Completeness details we're building in from the start

- **File type validation** — restrict to jpg / png / webp.
- **File size cap** — roughly 1–2MB; logos don't need to be larger.
- **Cache-busting via unique keys** — every upload writes to a *new* key (the uuid) instead of overwriting the old one. `logoUrl` just points at the new URL — no CDN cache-invalidation call ever needed.
- **Private bucket, CDN-only public access** — already true by default; the bucket itself was never made publicly listable, only reachable through the custom domain.
- **Least-privilege credentials** — already done via the scoped API token from step 2.

Why this design generalizes

> If we later decide customers can upload images too (or reuse this for `Coupon.imageUrl`, or a future avatar feature), **none of the storage/CDN plumbing changes**. The only thing that changes is which guard sits in front of the presign endpoint, and possibly the key prefix used. That's exactly why it was safe to build this once now, before deciding who's allowed to use it.

5. Glossary

Origin

The actual storage location holding a file — here, the R2 bucket.

CDN (Content Delivery Network)

A network of geographically distributed servers that cache copies of files close to users, cutting latency and shielding the origin from repeat traffic.

Edge location

One of the CDN's physical cache servers, spread around the world.

Cache hit / miss

Hit: the file was already cached at the nearest edge. Miss: it wasn't, so the edge had to fetch it from the origin first.

TTL (Time To Live)

How long a cached copy is considered valid before the edge re-checks it against the origin.

Egress

Data transferred out of a storage provider to the internet — the metric most cloud storage bills scale with as traffic grows.

Presigned URL

A temporary, cryptographically-signed URL granting permission to upload or download one specific file for a short window, without ever sharing the real credentials.

S3-compatible API

An interface that mirrors AWS S3's API so the same client libraries and tooling work unmodified against other providers — like R2.